Technical report — NLI over Swiss federal voting booklets with Apertus

What we built, how it works, and what the numbers say. Every number comes from a recorded run in `docs/runs/` (short names in brackets, listed under References) on the dev split, the validation set (val), the probe set or all three; **the sealed test dates have not been run**.

- **Track:** `Track 2A — OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets`
- **Event:** Online
- **Project:** TODO project name
- **Team:** TODO team name — Rashad, Kaan (TODO full names)
- **Demo:** none

1. Summary

Given a claim in German, French or Italian and a source from an official Swiss federal voting booklet, the system decides whether the source supports (entailment, 0), does not settle (neutral, 1) or refutes (contradiction, 2) the claim. The source is a passage (task B) or a whole booklet PDF (task A). **Apertus v1.5 makes every decision.** Everything else is local and deterministic and runs on CPU in the Docker image: reading the PDF, choosing the part of the booklet Apertus reads, and turning the paragraphs Apertus cites into evidence quoted verbatim with their 1-based PDF page.

The central experiment, full booklet against selected context, decided the design: in one interleaved run on 100 dev cases [E1-curve] the whole booklet scored 0.858 with 37,586 input tokens per case, the part of the vote that the claim names (`section-route`) 0.989 with 1,248; without any booklet text Apertus scores 0.435 [E2-cb]. Session 10 then optimised what the organisers score besides Macro-F1: tokens and *local time*.

Final defaults, published image [FIN]	**dev**	**val**	**probe (2 unseen dates)**
Task A Macro-F1 (weight 0.30 of the score)	**0.983**	**0.959**	**0.960**
Task A evidence (0.12)	**0.980**	**0.955**	**0.908**
Task A tokens per case, input + output (K, 0.09)	1,238 + 14.5	1,206 + 14.5	1,232 + 14.4
Task A local time per case, model at 1.0 / 0.3 s per request (T, 0.09)	0.017 s / 0.154 s	0.012 s / 0.034 s	0.038 s / 0.248 s
Task B Macro-F1 (0.28)	**0.967**	**0.961**	**0.971**
Task B tokens per case, input + output (K, 0.06)	1,994 + 8.4	1,957 + 9.0	1,831 + 8.3
Task B local time per case, 1.0 / 0.3 s per request (T, 0.06)	0.010 s / 0.0068 s	0.0069 s / 0.0052 s	0.014 s / 0.014 s
Labels written by Python (neutral fallback)	3 of 600	3 of 1,160	0 of 188
REF SCORE at 1.0 / 0.3 s per request (section 2)	**0.870 / 0.824**	**0.873 / 0.888**	**0.850 / 0.777**

Sealed test dates (2020-02-09, 2022-02-13, 2022-05-15): TODO, results after the organisers' evaluation.

2. What is scored, and what we optimise

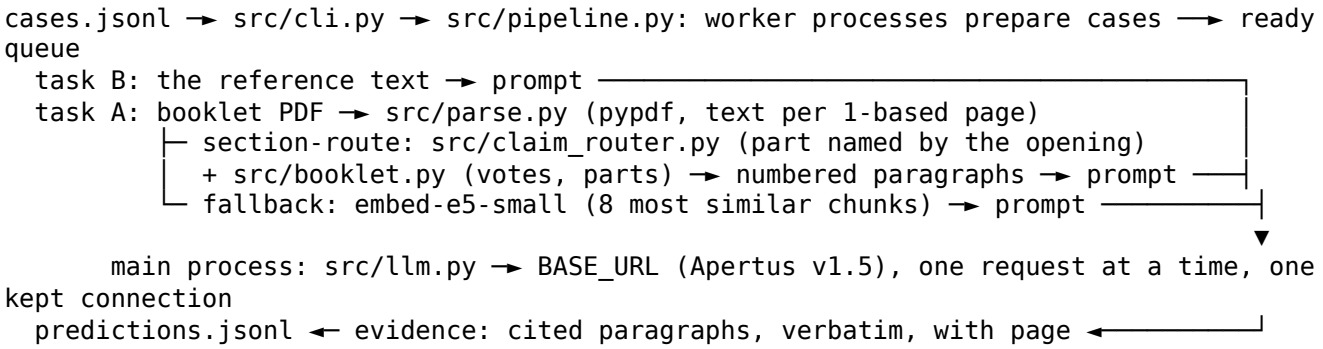
The organisers' score (Q&A of 8 October; `docs/official_scoring.md`): **Score = 0.4 S_B + 0.6 S_A**, with S_A = 0.50 Macro-F1 + 0.20 evidence + 0.15 T + 0.15 K and S_B = 0.70 Macro-F1 + 0.15 T + 0.15 K. K = the best team's tokens per case ÷ ours; T = the best local time per case ÷ ours, where **local time is the container's wall-clock time minus the time during which a model request is in flight**. The model's latency is free; start-up, PDF reading, embedding and every gap between two requests count. Task A and task B run in separate containers from the image we publish on ghcr.io.

To rank our own versions we use **REF SCORE** (`scripts/official_score.py`): the same formula with reference "best" values fixed by us before any measurement (the other teams' are unknown) (600 / 800 tokens and 0.05 / 0.005 s local time per case for task A / B; ratios capped at 1). Local time is measured the organisers' way (`scripts/local_time.py`): the image runs the organisers' command with 2 CPUs and 4 GB against a fake model that replays saved real answers after a fixed delay, 1.0 s (about Public AI's speed) and 0.3 s (a fast server), and logs when each request is in flight.

dev	`main` before session 10, 1.0 s [MAIN]	final, 1.0 s	`main`, 0.3 s	final, 0.3 s
Task A local time per case	0.803 s	0.017 s	0.742 s	0.154 s
T_A	0.062	1.000	0.067	0.325
K_A (tokens 1,252 both)	0.479	0.479	0.479	0.479
S_A	0.767	0.909	0.768	0.808
Task B local time per case	0.985 s	0.010 s	0.939 s	0.0068 s
T_B	0.005	0.500	0.005	0.735
K_B (tokens 1,240 → 2,003)	0.645	0.400	0.645	0.400
S_B	0.774	0.811	0.774	0.847
REF SCORE	**0.770**	**0.870**	**0.770**	**0.824**

Macro-F1 and evidence: `main` from session 9's final dev run (task A 0.980 / 0.980, task B with B-cut 0.967), final from [FIN] (task A 0.983 / 0.980, task B 0.967).

3. Architecture



- `src/cli.py`: the official entrypoint. Each case is split into `prepare()` (all local work) and `complete()` (the model call and the response). Answers every request that has an id, never stops on one bad case (fallback: neutral, no evidence), writes each answer at once, exit code 0.
- `src/pipeline.py` (session 10): two forked worker processes ("lanes"; one on a single CPU) prepare the cases while the main process sends the prepared ones in the order they become ready. Each lane owns whole booklets, so no PDF is read twice; cases that need no local model

are prepared first, task A cases grouped by booklet. Results do not depend on order or timing: every cache is keyed by content, and a check replays the saved answers of all 1,180 dev and val requests and finds every request, label and evidence unchanged (G1, G2).

- `src/llm.py`: the only module that calls a model: OpenAI-style `chat/completions` to `BASE_URL` with `API_KEY`, model from `MODEL`, temperature 0, over one kept connection (a new connection cost 486 ms per call through our proxy, a kept one 1.2 ms [CONN]). One retry for HTTP 5xx, timeouts and a lost connection, two for HTTP 429; if the endpoint refuses `response_format` or the model name, one fallback each.

3.1 How document context is prepared and supplied to the model

Parsing. pypdf reads the PDF locally and keeps each page's text with its 1-based PDF page number (the *n*-th page of the file). Cached in memory and in `/tmp` under the file's SHA-256.

Selection (`section-route`, task A). (1) *Router:* the claims open with a formula naming their source ("Laut der Zusammenfassung …", "Le Conseil fédéral estime …", "Il comitato afferma …"); regular expressions map the opening to one of five parts: summary, Federal Council, committee, text put to the vote, detailed explanation. (2) *Parser:* the table of contents gives each vote's sections; every start page is checked against its printed heading; the arguments are split by voice. (3) *Vote match:* the case's `vote` against the contents titles (fuzzy); if that fails, by meaning with multilingual-e5-small when the closest title leads clearly (session 10, for vote names in another language). (4) *Paragraphs:* the part's lines grouped into paragraphs, each with its page; a part over 8,000 characters keeps its 8 paragraphs most similar to the claim (e5). (5) *Fallback:* no route, no vote, or any error → the 8 chunks of the booklet most similar to the claim (e5).

What the prompt contains. System: the instruction and the decision rule (`A-v4-section-route-L1`):

> *Decide in this order: 1. Does the reference text deal with the subject of the claim at all? If not, the label is 1 (neutral). 2. If it does: 0 (entailment) if the reference text supports the claim; 2 (contradiction) only if the reference text states something that cannot be true together with the claim; otherwise 1 (neutral: insufficient information). Missing information is never a contradiction. […] A claim that gives a different number, share, date, actor or direction than the reference text gives for the same thing is a contradiction.*

User: `PART:` (which part, whose voice), `REFERENCE TEXT:` with paragraphs `[1] … [n]`, `VOTE:`, `CLAIM:`. Nothing is translated. `response_format: json_schema` forces `{"paragraphs": [≤ 3 numbers], "label": 0|1|2}`. Task B sends the same rule (`v3-topic-first`) with the whole reference and asks for `{"label": n}`.

Evidence. Only what Apertus cited: the cited paragraphs verbatim with their page, then the two halves of each (cut at the sentence end nearest the middle) up to five items. Neutral answers and task B carry none.

3.2 Who does what

Step	Python (rules, deterministic)	Local model (multilingual-e5-small, CPU)	Apertus v1.5
Read input, answer every id	✓		
PDF text per page	✓ (pypdf)		
Which part the claim names	✓ (regular expressions)		
Which vote of the booklet	✓ (title match)	only if the title match fails	
Which paragraphs	✓ (the part)	only parts over 8,000 characters, and the fallback	
Label	only the neutral fallback for a failed call or an unreadable answer: 6 of 2,128		✓ **every label**

Step	Python (rules, deterministic)	Local model (multilingual-e5-small, CPU)	Apertus v1.5
	final labels (task B), 0 in task A		
Which paragraphs justify it			✓
Evidence text and page	✓ (copies what Apertus cited)		

4. Use of Apertus

- **Model:** Apertus v1.5, named by `MODEL` (default `swiss-ai/Apertus-v1.5-8B`). Our runs used `swiss-ai/apertus-v1.5-8b` on Public AI. Which model and server the evaluation uses is not known to us.
- **How:** inference only, zero-shot, one call per case, temperature 0; answer budget 128 tokens (task A), 32 (task B). No fine-tuning. Reached only through `BASE_URL`; no key in the image.
- **Labels:** the dataset README now defines them ("The entailment class of the `claim` relative to the `reference_string` and voting booklet as a whole. May be `Entailment` (`0`), `Unrelated / Neutral` (`1`) or `Contradiction` (`2`)"). The prompts use the official guide's wording, which agrees; a shorter prompt with the README's names lost 0.31 Macro-F1 (section 6).
- **Development tools:** Claude Code wrote code and ran experiments; it is never called by the pipeline.

5. Data

- **Dataset:** `OSTswiss/MNLIoverSwissVotingBooklets` v1.1 (commit `9ff08597`, MIT): 1,488 rows, 335 exact duplicates; each row gives one task A and one task B case.
- **Splits** (by voting date, seed 42, duplicates removed): test 5 dates, 267 rows; dev 300 rows of the other 15 dates; val the 580 remaining rows of the dev dates (never used to change a rule). **Probe:** the 94 test rows of two of the five test dates (2021-03-07, 2025-09-28), opened in session 10 as unseen booklets; the other three test dates stay sealed. **On-topic neutral set:** 46 dev cases with the gold passage removed from what the model reads (expected neutral; indicative only).
- **Booklets:** the Federal Chancellery's PDFs, downloaded by script and not committed (one example booklet for `make run`). Swiss copyright law exempts official reports of authorities (Art. 5 URG). No personal data.

6. Evaluation

Method. The starter's `evaluate.py`, unchanged: Macro-F1 per task; for task A the share of gold entailment/contradiction cases where one of the first five evidence items matches the gold passage. The endpoint drifts, so **only arms of one interleaved run compare** (every case answered by all versions back to back, the backend of every answer recorded; all session 10 answers came from one backend). Rules were fixed before the runs; a version that fails stays in the code, switched off.

Version (step) [run]	What changes	Macro-F1 today → version: dev; val	Tokens per case (in + out)	Rule, outcome
Pipelining, kept connection, two lanes (2) [PIPE, LANES]	where and when local work runs	same answers (G1, G2)	same	local time falls: **adopted**
Whole reference for task B (2) [PIPE]	B-cut (session 9) off	0.967 → 0.967; 0.961 → 0.957 (session 9)	1,240 → 2,003	higher REF SCORE, mean over both delays: **adopted**

Version (step) [run]	What changes	Macro-F1 today → version: dev; val	Tokens per case (in + out)	Rule, outcome
P-short / P-short+K4 (5) [5A]	task A prompt in 200 tokens	0.980 → 0.670 / 0.619	1,252 → 1,123 / 694	TOKEN RULE: fail
K4 (5) [5A]	4 paragraphs instead of 8	0.980 → 0.990; 0.959 → 0.944	1,252 → 823	passes dev, fails val
BP-short / BP-short+B-cut4 (5) [5B]	task B prompt in 144 tokens	0.967 → 0.667 / 0.718 (29 / 24 unreadable)	2,003 → 1,942 / 831	fail
B-cut4 (5) [5B]	long reference → first line + 4 paragraphs	0.967 → 0.987; 0.957 → 0.961	2,003 → 892 / 901	passes TOKEN RULE; lowers REF SCORE (e5 local time): not adopted
CS (6) [6]	coupled schema: neutral cites nothing	0.983 → 0.983; on-topic neutral 31 → 28 of 46	same	fails (on-topic neutral share)
SL (6) [6]	second look when neutral cites	0.983 → 0.987; 0.959 → 0.962 (3 fixes, 0 breaks)	+0.3 % / +0.8 %	PAIRED RULE (+0.005 on val, sign test): fail
No `response_format` (7) [7]	answer read as free text	0.969 → 0.980 on 100 cases; 0 unreadable	+1.3 output	reader needs no change
Vote names by meaning (8)	e5 when the title match fails	same answers (G1, G2)	same	**adopted**
B-U (9) [9]	task B as task A's paragraphs	0.967 → 0.976	2,003 → 1,417 (+5.6 output)	adds output tokens and local time: not adopted

Probe: two unseen voting dates (test rows of 2021-03-07 and 2025-09-28, `data/probe/`, opened in session 10 before any change for them). Through the published image [FIN-probe]: task A **0.960**, evidence **0.908** (59 of 65), task B **0.971**, REF SCORE 0.850 / 0.777: val's level. 83 of 94 task A cases are routed; 11 fall back, 8 because the parser does not find the first vote of `2025_09_28_fr` (a contents entry without arguments), which also costs 3 of the 6 evidence misses. Reported, not fixed: a parser change needs its own check on dev and val.

Local time. The organisers count everything except the time a request is in flight. On `main` before session 10, each case was read, embedded and then sent: task A spent 0.803 s per case outside requests (the first case of each booklet waited for its PDF to be read), task B 0.985 s (B-cut's e5 embedding) [MAIN]. Now worker processes prepare the next cases while a request is in flight, the booklets are split between two of them, one connection is kept (a new one cost 486 ms per call through our proxy [CONN]), and task B sends the whole reference. Measured on the published image (2 CPUs, 4 GB): task A 0.017 s per case at 1.0 s per request and 0.154 s at 0.3 s, task B 0.010 s / 0.0068 s; peak memory 2.3 GiB (task A). What is left is the time to the first request (container, Python and, for task A, the first booklet: 3.0 s for task A, 0.9–1.8 s for task B in two runs of the same image), shared by all cases of a run (so val's 580 cases show lower values per case), and, with a fast model, PDF reading that no longer hides behind the requests.

Errors. On val, all 12 of E6's routed errors had the gold passage in front of the model (the 13th fell back): reading errors, not search errors. The remaining ones are mostly contradictions answered neutral and neutral claims about another ballot answered contradiction (task B). A Python rule "neutral with a citation → contradiction" would gain +0.014 on val but turns all 3 of its changes on the on-topic neutral set wrong; it is scored offline only and is not in the pipeline (every label comes from Apertus).

7. Limitations

- **Tuned on dev.** Prompts, router patterns and parser checks were written with dev in view; val confirms; the probe (two unseen dates) lands at val's level; three test dates and the private set are unseen.
- **`section-route` depends on the claims' openings and the booklet layout** (2018–2026 checked). A claim or vote that does not fit falls back (answered, less accurate). On the probe, one booklet's first vote fails a parser check (8 of 94 cases fall back).
- **Vote names by meaning** place a vote name in another language right or not at all on dev and val, but can place the name of a vote that is not in the booklet (12 % of such names); the dataset never asks that.
- **Endpoint and speed.** Public AI served the same model name from two backends that answer differently; our scores hold for the one that answered. Local time depends on how fast the evaluation's model answers: with a fast model (0.3 s per request) task A's PDF reading no longer hides behind the requests.
- **Not political advice.** The system checks a claim against the booklet's text only. Its answers are not voting recommendations. Every task A label 0 or 2 comes with the booklet page and the verbatim text it rests on.

8. Reproducibility

- **Published image:** `docker pull ghcr.io/rashad18236/hackapertus-voting-nli:latest` (built from `main` by `.github/workflows/publish-image.yml`, which also pulls it on a fresh runner and runs the organisers' command without internet); then the organisers' command with `BASE_URL` and `API_KEY` (README).
- **From source:** `make run` builds the image and answers `examples/cases.jsonl`; own cases with `make run CASES=… BOOKLETS=… OUTPUT_DIR=…`.
- **Hardware:** CPU only; 2 CPUs and 4 GB measured (peak memory 2.3 GiB with two worker processes).
- **Determinism:** temperature 0, local steps deterministic, caches keyed by content. A fake model (`scripts/stub_llm.py`) replays saved answers by request hash: every change to `src/` is checked against the requests, labels and evidence of all 1,180 dev and val cases (G1, G2) and by unit, contract and pipeline tests.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. The repository's `LICENSE` file is Apache-2.0; the team has yet to make the two agree.

References

- Dataset: OSTswiss/MNLIoverSwissVotingBooklets v1.1, Hugging Face (MIT). Starter: `gitlab.com/ifsoftware/hackapertus-starter`. Apertus v1.5, Swiss AI Initiative.
- Wang, L. et al. (2024). *Multilingual E5 Text Embeddings: A Technical Report.* arXiv:2402.05672 (`intfloat/multilingual-e5-small`, MIT). pypdf (BSD-3), rapidfuzz (MIT), onnxruntime (MIT), requests (Apache-2.0).
- Runs (`docs/runs/2026-10-12_rashad_` + name): FIN `final-defaults_{dev600,val1160}`, FIN-probe `final-defaults_probe188`; MAIN `local-time-main_devA300-devB300`; CONN `connection-time_devB20`; PIPE `pipelining_devA300-devB300`; LANES `lanes_devA300-devB300`; 5A `fewer-tokens-A_*`; 5B `fewer-tokens-B_*`; 6 `reading-A_*`; 7 `response-format-off_devA100`; 9 `unified-B_devB300`; replay reference `prompt-snapshot-final-s10_devAB-valA`. From 2026-10-09: E1-curve `context-curve_devA100`, E2-cb `closed-book_devA300`.